Аннотация

Мы критически анализируем MacroHFT, иерархическую RL-систему для минутной криптовалютной торговли, и частично воспроизводим её результаты на опубликованных ETH-экспертах. Мы рассматриваем допущения торгового симулятора, оцениваем, насколько обосновано применение RL, и анализируем специализацию по тренду и волатильности, динамическое смешивание Q-оценок и эпизодическую память.

Симулятор исполняет сделки без задержек, проскальзывания и market impact; это ограничивает переносимость результатов на реальную торговлю. Математические контрпримеры показывают, что классификация по тренду и волатильности не определяет оптимальное торговое действие. Мягкое смешивание Q-оценок само по себе не обеспечивает более быстрой адаптации. Авторы отдельно не проверяют, даёт ли волатильностная специализация преимущество перед специализацией только по тренду, а мягкое смешивание — перед сопоставимым hard routing. Скорость реакции на стрессовые события они не измеряют.

На ETH test два трендовых эксперта почти постоянно выбирают одно действие: один — flat, другой — long. Фиксированный эксперт, выбранный по доходности на validation, превосходит все исследованные нами агрегаторы на данном тесте, однако результат зависит от критерия выбора. Ridge-Static ($V_1 \rightarrow V_2$) получает 13.22% по авторской нормировке. Это близко к опубликованным 12.73% у MacroHFT без памяти, но существенно ниже результата полной модели (39.28%). Мы работаем с фиксированными низкоуровневыми RL-экспертами и не воспроизводим полную систему напрямую. Результаты не обосновывают необходимость режимной иерархии и дополнительной RL-агрегации. Вопрос о пользе RL относительно более простых торговых моделей остаётся открытым.

Наиболее перспективным направлением дальнейшего исследования считаем механизм памяти. Опубликованная абляция показывает, что он повышает доходность и Sharpe и снижает максимальную просадку на всех четырёх активах. Остаётся установить, связано ли это с использованием редких событий, сглаживанием Q-оценок или снижением риска. Также нужно проверить, сохраняется ли преимущество перед существующими альтернативами в более реалистичных торговых средах.

1 Постановка торговой задачи

1.1 Торговая среда и допустимые действия

Zong et al. [1] рассматривают торговлю одним криптоактивом на фиксированном минутном временном шаге. Задачу они формулируют как марковский процесс принятия решений:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, r, \gamma), \tag{1}$$

где $\mathcal{S}$ — множество состояний, $\mathcal{A}$ — множество действий, $\mathcal{T}$ — функция переходов, r — вознаграждение, а $\gamma \in [0, 1)$ — коэффициент дисконтирования.

Агенту доступны два действия: остаться без позиции (*flat*) или держать длинную позицию фиксированного размера (*long*). Обозначим текущую позицию перед решением через $P_t \in \{0, 1\}$, а выбранную целевую позицию — через a_t:

$$a_t \in \mathcal{A} = \{0, 1\}, \qquad P_{t+1} = a_t. \tag{2}$$

Значение 1 соответствует владению m единицами криптовалюты, значение 0 — отсутствию позиции. Если $a_t \neq P_t$, симулятор исполняет покупку или продажу фиксированного объёма; иначе сделки нет. Иными словами, рассматриваются только *taker-only* стратегии: позиция меняется агрессивной сделкой по цене, принятой в симуляторе. Снимки книги лимитных заявок служат признаками, но механизмов исполнения лимитных ордеров постановка не включает.

Пусть p_t обозначает цену закрытия минутного интервала, а m — фиксированный объём позиции. Авторы определяют вознаграждение как изменение стоимости портфеля после учёта транзакционных издержек:

$$r_t = m\Big[a_t(p_{t+1} - p_t) - \delta_t |a_t - P_t|\Big]. \tag{3}$$

Здесь δ_t — стоимость переключения позиции, выраженная в единицах цены за единицу актива. При пропорциональной комиссии c её можно записать как $\delta_t = c\,p_t$; в экспериментах ставка равна $c = 0.0002$, т.е. 0.02% от стоимости сделки [1]. Если позиция не меняется, симулятор не списывает комиссию.

Пространство *самих торговых действий* здесь существенно уже, чем в задачах оптимального исполнения и маркет-мейкинга. Каждую минуту агенту достаточно решить, держать ли фиксированный long или оставаться в cash. Однако оптимальная политика от этого не становится тривиальной.

1.2 Состояние и критерий оптимизации

Входные наблюдения включают OHLCV и пять уровней книги лимитных заявок. Авторы выделяют признаки текущего снимка рынка и контекстные признаки, вычисленные по предыдущим наблюдениям. На верхнем уровне авторы добавляют скользящие оценки тренда и волатильности, описывающие более длинный рыночный контекст (все признаки приведены в таблице 4 статьи [1]).

Стандартная цель RL-политики — максимизировать ожидаемое дисконтированное вознаграждение:

$$J(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} \gamma^t r_t\right], \tag{4}$$

где $\pi(a \mid s)$ задаёт распределение действий в состоянии s. В финансовой интерпретации вознаграждение (3) соответствует накоплению торгового PnL с комиссией. Для оценки результатов авторы также используют доходность, Sharpe ratio и максимальную просадку.

1.3 Ограничения модели исполнения

В принятой постановке авторы оптимизируют торговую политику π на исторической траектории цен, которая не зависит от её действий. При переходе из flat в long или обратно симулятор изменяет позицию, списывает комиссию и продолжает воспроизводить ту же последовательность рыночных наблюдений. Симулятор исполняет сделку по заданной цене независимо от объёма, состояния стакана и времени между сигналом и исполнением.

В реальной торговле эти условия могут не выполняться. Рассмотрим отдельно задержки, ликвидность и влияние собственных сделок на дальнейшее поведение рынка.

Задержки и проскальзывание. Торговый алгоритм принимает решение по информации от биржи. Между рыночным событием и исполнением ответного приказа проходит некоторое время. Данные должны поступить на сервер стратегии, алгоритм — обработать их, приказ — достичь биржи, а торговая система биржи — принять и исполнить его. Обозначим совокупную задержку через

$$\tau = \tau_{\text{data}} + \tau_{\text{calc}} + \tau_{\text{network}} + \tau_{\text{exchange}}. \tag{5}$$

Для сокращения сетевой составляющей профессиональные участники используют *коллокацию* (colocation): размещают торговые серверы в том же дата-центре, что и инфраструктура биржи, или рядом с ним. Это сокращает задержку передачи данных

и приказов и делает её стабильнее. При торговле с удалённого сервера сигнал проходит через дополнительные сетевые узлы; задержка становится больше и менее предсказуемой. Однако коллокация не устраняет время вычислений и обработки приказа биржей.

За время τ доступная цена может измениться. Например, алгоритм обнаруживает сигнал к покупке по котировке p_t^{ask}, но его рыночный приказ исполняется уже по $p_{t+\tau}^{\text{ask}}$. Разность

$$\varepsilon_t^{\text{lat}} = p_{t+\tau}^{\text{ask}} - p_t^{\text{ask}} \tag{6}$$

показывает проскальзывание покупки из-за задержки. Оно не обязательно положительно: цена может успеть измениться и в пользу трейдера. Однако сигнал часто возникает из-за события, на которое реагируют и другие участники. Тогда опоздавший приказ может систематически получать худшую цену, чем та, на которую рассчитывали при оценке дохода.

Задержка сильнее влияет на стратегии, принимающие решения каждую секунду или чаще. Если предсказуемое движение длится несколько сотен миллисекунд, даже качественный прогноз может потерять практическую ценность до момента исполнения. EarnHFT [2] рассматривает секундные торговые решения, но не включает подобные задержки в модель исполнения.

Переход MacroHFT [1] к минутному шагу снижает чувствительность некоторых стратегий к миллисекундным задержкам, но не устраняет её. Решение, принятое раз в минуту, может зависеть от кратковременного изменения котировок в конце этой минуты. Кроме того, если OHLCV за интервал становятся доступны только после его завершения, исполнение по цене закрытия того же интервала требует отдельного обоснования. Чтобы оценить влияние задержек на доходность конкретных минутных политик, нужно моделировать исполнение.

Глубина стакана и market impact. Даже при нулевой задержке рыночный приказ не обязательно исполняется целиком по лучшей доступной цене.

Книга лимитных заявок содержит цены и объёмы, по которым другие участники согласны купить или продать актив. Лучший ask — минимальная цена продажи, лучший bid — максимальная цена покупки. Разность между ними — *bid–ask spread*, одна из составляющих стоимости немедленного исполнения. Покупающий по рынку платит ask, а продающий получает bid.

На лучшем ценовом уровне находится конечный объём. Допустим, стратегия отправляет рыночную заявку на покупку 0.2 ETH, а по лучшей цене выставлено только 0.05 ETH. Оставшиеся 0.15 ETH биржа исполняет на следующих, более дорогих уровнях стакана. Средняя цена исполнения составит

$$\bar{p}_t^{\text{buy}}(q) = \frac{1}{q}\sum_{k=1}^{K} p_{t,k}^{\text{ask}} q_{t,k}, \qquad \sum_{k=1}^{K} q_{t,k} = q, \tag{7}$$

где $q_{t,k}$ — объём, фактически исполненный на k-м уровне. Если одной лучшей котировки недостаточно, средняя цена покупки оказывается выше лучшего ask. При продаже возникает аналогичный эффект на стороне bid.

Проскальзывание возможно даже без сетевой задержки: его создаёт ограниченная глубина стакана. Чем больше объём рыночного приказа относительно доступной ликвидности, тем больше уровней он может пройти.

Авторы MacroHFT строят признаки по снимкам пяти уровней стакана, но цену исполнения в функции вознаграждения задают через минутные цены закрытия и фиксированные транзакционные издержки. В частности, формула (3) не учитывает стоимость прохождения нескольких уровней стакана. Цена закрытия не обязана совпадать с лучшим bid или ask на момент исполнения.

Прохождение стакана связано с более широкой проблемой *market impact* — влиянием собственной сделки на рыночную цену. Крупный рыночный приказ потребляет доступную ликвидность, может сдвинуть лучшие котировки и изменить решения других участников. Маркет-мейкеры могут снять заявки, уменьшить выставленные объёмы или пересмотреть цены, ожидая дальнейшего движения. По мере восстановления ликвидности часть воздействия может исчезнуть, а часть — сохраниться. На различии временного и постоянного воздействия, в частности, основаны модели оптимального исполнения Almgren–Chriss [6].

Исторический симулятор не моделирует этот механизм. Рыночная компонента следующего состояния не зависит от действия исследуемой стратегии:

$$\widehat{\mathcal{T}}(s_{t+1}^{\text{mkt}} \mid s_t, a_t) = \widehat{\mathcal{T}}(s_{t+1}^{\text{mkt}} \mid s_t). \tag{8}$$

В реальности распределение будущих котировок может зависеть от направления и объёма собственного приказа, а также от реакции на него других участников. Исторические данные позволяют оценить динамику рынка без нашего вмешательства, но не обязательно описывают её после вмешательства.

Это различие становится существенным при росте капитала. В модели (3) доходность в денежном выражении масштабируется линейно с размером позиции m. В реальности увеличение m может сильнее ухудшать цену исполнения, менять последующую ликвидность и тем самым нарушать линейное масштабирование прибыли. Стратегия, прибыльная при торговле небольшим объёмом, может оказаться убыточной при попытке вложить в неё значительно больший капитал.

Для фиксированного объёма 0.2 ETH на ликвидном рынке влияние отдельной сделки может быть небольшим. Но результаты симулятора не позволяют оценить ни величину этого эффекта, ни масштаб капитала, при котором преимущество политики сохраняется.

Ограничение пространства торговых стратегий. В постановке (2) агент может изменить позицию только агрессивной сделкой. Такой способ исполнения позволяет относительно быстро получить нужную позицию, но требует принять предложенную рынком цену и оплатить taker-комиссию.

Участник рынка может действовать иначе: выставить лимитный ордер на покупку по bid или на продажу по ask и ждать встречного приказа. В этом случае он *предоставляет ликвидность* рынку (maker), а не потребляет её (taker). При удачном исполнении последовательных покупки и продажи такой участник может получить разницу между bid и ask вместо того, чтобы дважды её заплатить. На некоторых биржах и тарифных уровнях действуют *maker rebates* — выплаты или скидки за предоставленную ликвидность.

Однако пассивное исполнение — не бесплатная замена рыночному приказу. Лимитный ордер может вообще не исполниться: цена уйдёт в нужном направлении, не достигнув выставленной заявки. На одном ценовом уровне может находиться очередь более ранних приказов, поэтому касания цены недостаточно для исполнения конкретной заявки. Наконец, пассивный ордер особенно часто исполняется тогда, когда информированные участники готовы торговать против него. Это создаёт риск *adverse selection*: например, покупка по bid исполняется непосредственно перед дальнейшим падением цены.

Поэтому высокочастотные стратегии предоставления ликвидности требуют отдельно управлять очередью заявок, их вре-

менем жизни, отменами, риском неисполнения и накопленной позицией. В то же время для стратегий с небольшим ожидаемым движением цены экономия на spread и комиссиях может определять саму возможность получения прибыли. Ограничение taker-only исключает такие решения, в том числе размещение нескольких лимитных заявок на разных ценовых уровнях и последовательное исполнение крупного ордера небольшими частями.

При комиссии $c = 0.0002$ покупка и последующая продажа уже требуют около четырёх базисных пунктов относительно торгового оборота, даже без учёта spread и проскальзывания. Если доступный прогностический сигнал не покрывает эти издержки, прибыльной активной taker-политики может не существовать. При тех же рыночных условиях стратегии пассивного исполнения остаются потенциально прибыльными. Поэтому поиск

$$\pi^*_{\text{taker}} = \arg\max_{\pi \in \Pi_{\text{taker}}} J(\pi)$$

не гарантирует, что будет найдена лучшая стратегия в более широком пространстве допустимых торговых действий Π_{all}.

Перенос результатов из симулятора. Описанные упрощения способны изменить не только абсолютную доходность стратегий, но и их сравнительную эффективность. Обозначим через $\widehat{J}(\pi)$ результат политики π в историческом симуляторе. Через $J_{\text{real}}(\pi)$ обозначим ожидаемый результат при реальном исполнении с тем же капиталом.

Из неравенства

$$\widehat{J}(\pi_A) > \widehat{J}(\pi_B)$$

в общем случае не следует

$$J_{\text{real}}(\pi_A) > J_{\text{real}}(\pi_B).$$

Например, пусть две политики на одном историческом периоде получили прибыль 10 и 8 базисных пунктов капитала после комиссий, учтённых симулятором. Первая совершила 20 сделок, вторая — две. Предположим, что реальное исполнение каждой сделки создаёт дополнительные потери, равные одному базисному пункту капитала. Тогда

$$\begin{aligned} \widehat{J}(\pi_A) &= 10, \quad J_{\text{real}}(\pi_A) = -10, \\ \widehat{J}(\pi_B) &= 8, \quad J_{\text{real}}(\pi_B) = 6. \end{aligned} \tag{9}$$

Числа условные. Пример показывает, что политика, имеющая преимущество в симуляторе, может уступить более осторожной политике после учёта исполнения. Для крупного объёма различие может быть ещё существеннее, поскольку дополнительные издержки уже не обязаны быть постоянными на сделку.

Рост доходности при добавлении архитектурного компонента RL устанавливает его полезность прежде всего в рассматриваемой модели торговой среды. Сохранение эффекта при задержках, ограниченной ликвидности и изменении объёма капитала остаётся самостоятельной экспериментальной задачей.

Возможные направления дальнейшего исследования. Первый подход — расширить математическую модель исполнения. Вместо фиксированной комиссии можно использовать функцию издержек, зависящую от объёма, spread, доступной глубины стакана и задержки:

$$C_t = C(s_t, q_t, \tau_t), \qquad q_t = m|a_t - P_t|. \tag{10}$$

Более полная модель может включать временное и постоянное воздействие сделки на цену, вероятность частичного исполнения и восстановление ликвидности. Её параметры оцениваются по данным книги заявок и фактических сделок. Такой подход позволяет исследовать чувствительность результатов к конкретным характеристикам исполнения при сравнительно небольших вычислительных затратах. Но функциональную форму воздействия приходится выбирать заранее, а реакция других участников рынка может плохо описываться простой параметрической зависимостью.

Второй подход — агентное моделирование (*agent-based modelling*, ABM). В подобной среде виртуальная биржа обрабатывает отдельные приказы, отмены и сделки, а множество торговых агентов формирует поток заявок. Исследуемая политика взаимодействует с ними через тот же механизм исполнения, поэтому её действия могут изменять стакан и будущие наблюдения.

Пример такой инфраструктуры — ABIDES [7], дискретно-событийный рыночный симулятор, который моделирует книгу заявок, взаимодействие участников и сетевые задержки. События могут иметь временные метки с наносекундным разрешением, что позволяет воспроизводить порядок поступления приказов на масштабах, намного меньших минуты. Наносекундная точность временной шкалы сама по себе не гарантирует правдоподобия поведения участников.

Основная трудность ABM — калибровка. Необходимо воспроизвести не только распределение доходностей, но и поток ордеров, глубину стакана, время жизни заявок, скорость восстановления ликвидности и реакции участников на крупные сделки. Такая среда требует значительно больше вычислений, чем воспроизведение исторических минутных наблюдений. Даже хорошо откалиброванный агентный рынок остаётся моделью, а не точной копией реальной биржи.

Оба направления позволяют конкретнее проверить практическую ценность торговых RL-архитектур: сохраняются ли преимущества политики π при изменении задержки τ, объёма позиции m и механизма исполнения? При сравнении MacroHFT с более простыми политиками такая проверка может изменить выводы, сделанные по одному историческому backtest, даже без изменения самих обученных моделей.

2 Архитектура MacroHFT: мотивация и проверка предложенных решений

MacroHFT [1] продолжает линию исследований RL в торговых задачах, представленную, в частности, в [3, 4, 5]. Непосредственный предшественник — EarnHFT [2]: система обучает пул низкоуровневых политик, а отдельный RL-маршрутизатор выбирает, какой агент будет управлять торговлей. Низкоуровневые решения принимаются каждую секунду, выбор агента — каждую минуту.

В MacroHFT оба уровня работают на минутном шаге. Авторы формируют пул из шести агентов, обученных на трендовых и волатильностных подвыборках, а верхняя сеть смешивает их Q-оценки вместо выбора одного эксперта. Далее проверим, какие проблемы должны решать эти изменения и какие эксперименты позволяют проверить предложенные объяснения.

2.1 Добавление волатильности к классификации рыночных режимов

Мотивация и архитектурное решение. В основе режимной специализации лежит предположение, что на разных участках рынка полезны разные торговые политики. MacroHFT уточняет эту идею: классификации только по тренду недостаточно, поскольку рынки с похожим направлением движения могут

иметь разную волатильность. Авторы связывают игнорирование этого различия с ухудшением специализации и управления риском [1].

Авторы разбивают исторические данные на блоки длины L. Для каждого блока они вычисляют наклон сглаженного ценового ряда и волатильность доходностей. Границы, определённые на train, задают три трендовые категории и три категории волатильности. В результате получаются подвыборки

$$\begin{aligned} &\mathcal{D}_1^{\mathrm{tr}},\ \mathcal{D}_2^{\mathrm{tr}},\ \mathcal{D}_3^{\mathrm{tr}},\\ &\mathcal{D}_1^{\mathrm{vol}},\ \mathcal{D}_2^{\mathrm{vol}},\ \mathcal{D}_3^{\mathrm{vol}}. \end{aligned} \tag{11}$$

Для каждой подвыборки авторы обучают отдельную dueling DDQN-сеть. Это две классификации по три категории, а не девять пересечений тренда и волатильности. Для ETH, DOT и LTC авторы задают $L = 4320$ минут, для BTC — $L = 360$ минут [1].

Что требуется обосновать. Различие статистических характеристик рынка само по себе не объясняет, почему на соответствующих подвыборках лучше обучать отдельные политики. Для торгового решения существенны условные последствия действия:

$$\Delta^*(s) = Q^*(s,1) - Q^*(s,0). \tag{12}$$

При бинарном выборе long/flat знак $\Delta^*(s)$ определяет действие, а его величина характеризует преимущество одного решения над другим. Разбиение по тренду и волатильности полезно для специализации, если оно упрощает оценивание таких различий. Авторы отдельно не проверяют, улучшает ли выбранная классификация качество обучения политики.

Проблему показывает простой контрпример. Рассмотрим две истории из пяти последовательных цен:

$$\begin{aligned} H_A &= p(1, k, k^2, k, 1),\\ H_B &= p(1, k^{-1}, k^{-2}, k^{-1}, 1), \qquad k > 1. \end{aligned} \tag{13}$$

Обе истории заканчиваются на исходной цене p. Благодаря симметрии наклон линейной регрессии цены по времени равен нулю в обоих случаях. Простые доходности составляют

$$(k-1,\ k-1,\ k^{-1}-1,\ k^{-1}-1)$$

для H_A и ту же последовательность в обратном порядке для H_B. У обеих историй совпадают средняя доходность и её стандартное отклонение.

Последние движения противоположны: в H_A цена снижается, в H_B растёт. Допустим, условная динамика следующей доходности u_{t+1} имеет положительную автокорреляцию:

$$\mathbb{E}[u_{t+1} \mid H_t] = \rho u_t, \qquad \rho > 0. \tag{14}$$

Тогда при исходной позиции flat агент в H_A сохраняет cash, а в H_B покупает, если ожидаемое движение покрывает стоимость входа. Одинаковые линейный тренд и волатильность здесь соответствуют разным предпочтительным действиям.

Этот пример использует линейный тренд без предварительного фильтра. Он показывает, какую информацию теряют сводные характеристики: порядок движений и условная предсказуемость не определяются их средним значением и разбросом. Сглаживание и изменение длины окна не заменяют проверки того, сохраняет ли выбранная классификация полезные для торгового решения различия.

Возможна и обратная ситуация. Два режима могут различаться дисперсией доходности, но иметь одинаковое условное ожидание. Тогда одноперiодная политика, максимизирующая ожидаемый PnL при одинаковых издержках, может выбирать одно действие в обоих режимах. Различие волатильностей само по себе не требует разных нейросетей или разных политик.

Таблица 1: Экспозиция трендовых экспертов на ETH test: доля минут в long, %. Каждый эксперт торгует по собственной последовательности позиций.

Эксперт	Bear	Neutral	Bull
`slope/1`	9.66	5.14	11.04
`slope/2`	26.44	26.71	49.96
`slope/3`	99.88	99.73	99.64

Недостающий контроль. Сравнение MacroHFT с EarnHFT не выделяет эффект добавления волатильности: одновременно меняются пул экспертов, временная структура иерархии, агрегация и обучение. В Table 3 [1] авторы исследуют другие компоненты, но не проводят отдельную абляцию trend-only против trend-plus-volatility.

Кроме того, переход от трёх экспертов к шести увеличивает число обученных моделей. Поэтому даже выигрыш такой системы ещё не устанавливает, что полезна именно волатильностная специализация. Нужен контроль с тем же числом экспертов, например с несколькими независимыми агентами на каждом трендовом подмножестве или с альтернативным разбиением сопоставимого размера.

Чтобы проверить эту идею, нужно разделить три эффекта: добавление информации о волатильности, изменение обучающей выборки и увеличение числа моделей. Без такого сравнения дополнительная ось специализации остаётся инженерной гипотезой с неустановленным вкладом в результат.

2.2 Какие политики создаёт раздельное обучение

От обучающего режима к поведению эксперта. После сегментации авторы обучают шесть политик $\pi_1, \ldots, \pi_6$. Каждая получает собственную подвыборку, но использует одни и те же действия $\mathcal{A} = \{0, 1\}$. Режим задаёт состав обучающих примеров; он не ограничивает область применения готовой сети.

Отсюда два разных вопроса. Получает ли агент сравнительное преимущество в назначенном ему режиме? И умеет ли он распознавать, когда условия изменились и прежнее действие перестало быть выгодным? Первое свойство не гарантирует второго.

В нашем воспроизведении опубликованные трендовые агенты на ETH показывают особенно наглядное различие. Таблица 1 содержит долю времени в long внутри ретроспективно размеченных Bear-, Neutral- и Bull-сегментов тестового периода.

Агент `slope/3` держит ETH почти постоянно, включая медвежьи сегменты. `slope/1`, напротив, преимущественно остаётся в cash независимо от тренда. На validation наблюдается та же общая картина: экспозиция этих агентов составляет соответственно 99.56% и 5.84%. Эта картина не ограничена одним необычным участком test.

Такие политики действительно могут выигрывать в собственных режимах. Почти постоянный long зарабатывает на растущем рынке, а почти постоянный flat избегает потерь на падающем. Однако этот результат можно получить без распознавания смены режима внутри эксперта: достаточно устойчивого предпочтения одной позиции.

Наше разложение условного PnL согласуется с этой интерпретацией: около 85–87% измеренного контраста относительной специализации крайних трендовых агентов связано с компонентом рыночной экспозиции. Это доля показателя специализации, а не общей прибыли агента.

На исследованных траекториях часть пула работает преимущественно как набор направленных предпочтений long/flat. Распознавать, когда доверять каждому такому предпочтению, должен верхний уровень. Само обучение бычьего эксперта на бычьих данных ещё не создаёт механизм выхода при переходе к медвежьему рынку.

Нужны ли для этого отдельные RL-сети. Для почти постоянной позиции сложная нейросеть не нужна. Но постоянство действия ещё не доказывает бесполезности её численных Q-оценок: даже при неизменном знаке преимущества long его величина может содержать информацию, полезную для гиперагента.

Поэтому нужна конкретная абляция: сохраняется ли качество системы, если заменить рекомендации крайних экспертов константными flat/long-политиками или убрать зависимость величины их Q-advantages от состояния? Она отделила бы полезность непрерывных оценок от полезности самого разнообразия позиций.

Результаты остальных экспертов также не подтверждают, что разбиение данных требует иерархии. `slope/2` получил 23.87% на всём ETH test по авторской нормировке доходности и превзошёл все рассмотренные нами агрегаторы. Это показывает, что полезная политика, обученная на одной подвыборке, может применяться значительно шире её исходной категории.

Причина такого результата требует отдельной проверки. Подвыборка могла упростить обучение, соответствующая политика могла лучше перенестись на новый период, а результат мог зависеть от конкретного запуска. Чтобы проверить, нужно ли раздельное обучение, нужен агент той же архитектуры, обученный на всём рынке, а также контроль с несколькими инициализациями. Единая политика с режимными признаками или балансировкой обучающих примеров — прямая альтернатива шести независимым сетям.

2.3 Замена выбора эксперта взвешиванием Q-оценок

Предлагаемое решение. В аннотации MacroHFT авторы называют решения отдельного агента «one-sided and highly biased». С этой особенностью они связывают возможные потери в экстремальных рыночных условиях [1] и предлагают одновременно учитывать оценки нескольких экспертов.

Обозначим через s_t доступное состояние, включая текущую позицию. Каждая сеть получает предназначенную ей часть этого состояния. Жёсткая маршрутизация имеет вид

$$\begin{aligned} i_t &= g(s_t), \\ \pi_{\text{hard}}(s_t) &= \arg\max_{a\in\{0,1\}} Q_{i_t}(s_t, a). \end{aligned} \tag{15}$$

Маршрутизатор g выбирает одного эксперта, после чего система исполняет его предпочтительное действие.

В MacroHFT верхняя сеть вместо индекса формирует softmax-веса:

$$w_i(s_t) = \frac{\exp f_i(s_t)}{\sum_{j=1}^{6} \exp f_j(s_t)}. \tag{16}$$

Итоговая Q-оценка задаётся суммой

$$Q_{\text{mix}}(s_t, a) = \sum_{i=1}^{6} w_i(s_t) Q_i(s_t, a). \tag{17}$$

Для двух действий достаточно разностей

$$d_i(s_t) = Q_i(s_t, 1) - Q_i(s_t, 0), \tag{18}$$

поэтому политика принимает вид

$$\pi_{\text{mix}}(s_t) = \mathbf{1}\left\{ \sum_{i=1}^{6} w_i(s_t) d_i(s_t) > 0 \right\}. \tag{19}$$

При равенстве политика выбирает flat.

Что действительно меняется. Формула (19) не диверсифицирует капитал между шестью стратегиями. На выходе остаётся одна позиция: 0 или m единиц актива. Мягкое смешивание изменяет правило выбора этой позиции, но не создаёт промежуточной экспозиции.

Более того, в бинарной задаче любое действие такой смеси совпадает с действием хотя бы одного эксперта:

$$\pi_{\text{mix}}(s) \in \{\pi_1(s), \dots, \pi_6(s)\}. \tag{20}$$

Если взвешенная сумма $d_i(s)$ положительна, хотя бы одно $d_i(s) > 0$. Если сумма неположительна, хотя бы один эксперт допускает flat. При доступе к той же информации достаточно выразительный hard router может выбрать эксперта с тем же действием.

Результат не устанавливает одинаковой сложности обучения двух механизмов. Однако он исключает объяснение, будто hard routing по своей природе не способен выдать нужное защитное действие, а soft mixing расширяет множество доступных решений. Искать преимущество следует в обучении и использовании оценок экспертов.

У численного смешивания есть собственная цена. Влияние эксперта определяет произведение $w_i d_i$, а не один лишь вес. Например,

$$w_1 = 0.9, \quad d_1 = 0.1, \qquad w_2 = 0.1, \quad d_2 = -2$$

дают суммарное преимущество -0.11 и решение flat. Эксперт с 10% веса определяет действие вопреки эксперту с 90%.

Q-advantage оценивает разницу будущих наград, а не вероятность правильности рекомендации. Большой модуль может отражать как экономически существенную разницу между действиями, так и ошибку оценки. Поэтому график весов сам по себе не показывает, кто определил действие. Для этого нужны как минимум произведения $w_i d_i$ и их сумма.

Положительное масштабирование Q-функции сохраняет политику отдельного эксперта, но способно изменить действие ансамбля при фиксированных весах. Качество численных оценок становится отдельным требованием, которого нет у простого голосования по знакам d_i. Авторы не исследуют этот вопрос через сравнение сырых Q-оценок, их нормировки и рекомендаций самих политик.

2.4 Заявленное ускорение адаптации не измерено

Исходная проблема. Во введении MacroHFT авторы называют запаздывающую реакцию отдельных политик на внезапные колебания одной из причин перехода к новой архитектуре. В разделе результатов они приписывают ей «ability to adjust quickly to sudden market changes» [1]. Уменьшает ли предложенная система задержку перехода к подходящему торговому действию при резком изменении рынка?

Суммарная доходность за несколько месяцев на этот вопрос не отвечает. Политика может лучше выбирать прибыльные периоды в спокойном рынке, но реагировать на обвалы так же медленно. Возможна и обратная ситуация: более ранние выходы снижают потери при падениях, но уменьшают доходность из-за ложных тревог.

Смешивание не устраняет задержку распознавания. Оба верхнеуровневых механизма могут обновлять решение на очередном минутном шаге. Если рыночные признаки не позволяют своевременно обнаружить изменение, эта проблема сохраняется и у обучаемых весов. Наличие подходящего эксперта в пуле тоже недостаточно: его рекомендация должна получить достаточное влияние на итоговое действие.

Это видно даже для двух экспертов. Пусть один предпочитает long, другой — flat:

$$d_{\text{long}} = u > 0, \qquad d_{\text{flat}} = -v < 0.$$

Обозначим вес второго через b_t. При выборе эксперта с наибольшим весом переход к flat происходит, когда $b_t > 1/2$. При смешивании Q-оценок условие выхода имеет вид

$$\begin{aligned} (1 - b_t)u - b_t v &\le 0, \\ b_t &\ge \frac{u}{u + v}. \end{aligned} \tag{21}$$

Если $v = u/4$, для выхода требуется уже 80% веса flat-эксперта. Если $v = 4u$, достаточно 20%. При одной и той же траектории изменения весов смесь может переключиться как позже, так и раньше жёсткого выбора. Ускорение реакции не следует из самой операции смешивания: оно зависит от оценок экспертов и обученного правила их использования.

Этот контрпример не описывает измеренное поведение MacroHFT. Он опровергает универсальное объяснение «несколько взвешенных экспертов реагируют быстрее, чем один выбранный».

Что представлено в статье. Figure 4 содержит отдельные примеры торговых решений, в том числе выход во время падения LTC. Но для того же события авторы не показывают реакцию EarnHFT, не анализируют систематическую выборку стрессовых эпизодов и не приводят распределение задержек выхода из позиции.

Figure 5 показывает веса экспертов, усреднённые за 60 минут. Такой график не показывает, переключились ли веса до резкого движения, через минуту после него или значительно позже в пределах того же часа. Кроме того, веса без Q-advantages не определяют момент изменения торговой позиции.

Даже сводные риск-метрики не показывают безусловного преимущества новой системы. В Table 1 максимальная просадка на DOT составляет 15.89% у MacroHFT против 5.80% у EarnHFT; на LTC — 14.24% против 9.63% [1]. Эти числа не измеряют задержку адаптации, но и не могут служить подтверждением того, что новая архитектура систематически лучше защищает от крупных потерь.

Авторы не проверяют заявленную связь между архитектурой и быстрой адаптацией. Они приводят итоговые торговые результаты и иллюстрации поведения, но не измеряют преимущество, которым мотивируют изменение механизма выбора политики.

Какой эксперимент проверял бы это утверждение. Hard-routing baseline в наборе уже есть: EarnHFT входит в число сравниваемых алгоритмов. Недостающий контроль — обучаемый hard router и обучаемый soft mixer на одном пуле зафиксированных экспертов, с одинаковыми входами, торговой частотой и сопоставимым обучением.

Такой эксперимент должен выделять стрессовые события по заранее заданному правилу. Затем нужно оценить задержку изменения позиции, потери в окне события и частоту ложных защитных выходов вне этих окон. Эксперимент должен явно учитывать начальную позицию и возможность сократить риск: агент, который ещё до события находился в cash, не совершал быстрого выхода.

Дополнительный контроль — заморозить веса смеси перед событием, сохранив изменение Q-оценок экспертов. Разница с динамическими весами показала бы, помогает ли именно перевзвешивание, или защитное действие обеспечивают сами низкоуровневые сети. Итоговый PnL всей системы не позволяет разделить эти механизмы.

2.5 Что показывают простые альтернативы агрегации

Проверяемая замена. В наших экспериментах мы фиксируем шесть опубликованных ETH-экспертов, авторские признаки, размер позиции и комиссию. Меняем только правило, которое преобразует оценки экспертов в торговое действие. Каждая стратегия передаёт экспертам собственную предыдущую позицию.

Помимо равномерной смеси, большинства голосов и нормированных Q-advantages, мы проверяем выбор одного фиксированного эксперта, табличный hard router и supervised-контроллер на основе Ridge. Табличный router мы обучаем на девяти сочетаниях уровней тренда и волатильности; он выбирает одного из шести экспертов. Это простая обучаемая альтернатива, но не воспроизведение RL-маршрутизатора EarnHFT.

Ridge также нельзя описывать только как замену softmax-весов постоянными коэффициентами. В ноутбуке Ridge прогнозирует следующую минутную доходность:

$$\widehat{\mu}_t = b + \sum_{i=1}^{6} \beta_i \frac{d_i(s_t)}{\sigma_i}, \tag{22}$$

где масштабы σ_i мы оцениваем на обучающем отрезке метамодели. После этого применяем правило с учётом комиссии:

$$\pi_{\text{Ridge}}(s_t) = \underset{a \in \{0,1\}}{\arg\max} \left\{ a\widehat{\mu}_t - c|a - P_t| \right\}. \tag{23}$$

Из flat оно входит при $\widehat{\mu}_t > c$, а из long выходит при $\widehat{\mu}_t \le -c$. Такое правило — самостоятельный простой способ управлять позицией поверх выходов RL-экспертов.

Основной вариант Ridge мы обучаем на первой половине validation (V_1), а регуляризацию выбираем на второй (V_2). Контекстная версия добавляет взаимодействия экспертных сигналов со скользящими трендом и волатильностью. Низкоуровневые политики в обоих случаях остаются неизменными.

Результат, который требует объяснения. На этом тесте лучший из рассмотренных вариантов использует эксперта `slope/2`, выбранного по validation return, и вообще не переключает экспертов. Его MTM Return составляет 23.28% против 9.08% у Majority. Экспозиция немного ниже (32.09% против 35.38%), а максимальная просадка почти совпадает (9.54% против 9.65%). Разрыв нельзя объяснить только большей долей времени в long.

В то же время результат зависит от процедуры выбора эксперта. Исходный выбор по validation Sharpe привёл к `vol/2` и отрицательной тестовой доходности. Наблюдение устанавливает лишь более узкий факт, а не надёжность фиксированного выбора вообще: среди опубликованных политик есть эксперт, который в этом эксперименте работает лучше всех наших комбинаций. Такой baseline необходим при проверке добавочной ценности агрегации.

Статическая Ridge получила 13.22% по авторской нормировке. В Table 3 авторы сообщают 12.73% для MacroHFT без памяти и 39.28% для полной модели [1]. Простая supervised-альтернатива достигает численно сопоставимого результата с опубликованной конфигурацией без памяти. Это основание включить её в прямое сравнение, а не приписывать весь результат полной системы одному обучаемому голосованию.

Другие проверки не поддержали предположение, что достаточно любого упрощения. Нормировка Q-advantages ухудшила

Таблица 2: Выбранные результаты альтернативных способов использования шести ETH-экспертов на test за июнь–октябрь 2023 года. MTM Return, Sharpe и MDD рассчитаны при общем начальном капитале; Author-style Return использует знаменатель `required_money`. Exposure — доля минут в long.

Метод	MTM, %	Author-style, %	Sharpe	MDD, %	Exposure, %
Uniform Q	5.72	5.14	0.773	12.92	29.64
Normalized Q	-4.04	-3.53	-0.333	18.07	34.31
Majority	9.08	9.31	1.128	9.65	35.38
Табличный hard router	10.04	9.94	0.915	18.17	82.93
Ridge-Static, $V_1 \to V_2$	13.36	13.22	1.414	12.96	46.80
Ridge-Contextual, $V_1 \to V_2$	6.88	7.22	0.992	9.37	24.67
Фиксированный эксперт: выбор по validation return	23.28	23.87	2.424	9.54	32.09
Фиксированный эксперт: выбор по validation Sharpe	-22.47	-18.74	-3.116	25.28	22.80

результат. Табличный hard router почти не превзошёл Majority по доходности, но существенно увеличил просадку и экспозицию. Контекстная Ridge уступила статической по доходности. Наши данные не подтверждают ни универсального преимущества hard routing, ни объяснения всех проблем неправильным масштабом Q-оценок.

2.6 Эпизодическая память при обучении гиперагента

2.6.1. Мотивация

После обучения специализированных экспертов и введения гиперагента остаётся проблема устойчивости его обучения. В разделе 4.3 MacroHFT [1] авторы выделяют две причины. Первая — большой разброс наград у похожих рыночных состояний. Вторая — редкие резкие колебания, которые обычный experience replay, по их предположению, использует недостаточно эффективно.

Experience replay уже позволяет сохранять прошлые переходы и повторно использовать их при обучении. Однако равномерная выборка воспроизводит частоты наблюдений: обычные состояния встречаются в обучающих пакетах значительно чаще редких. Если несколько эпизодов крупных потерь определяют качество всей стратегии, такое распределение обновлений может оказаться неудачным.

Авторы предлагают дополнить обучение эпизодической памятью: для текущего состояния находить похожие переходы и использовать сохранённые оценки их ценности как дополнительный обучающий сигнал. Ожидается, что это согласует Q-оценки близких состояний и поможет получить политику, быстрее реагирующую на резкие рыночные изменения.

Такой подход продолжает существующую линию исследований. В Episodic Memory Deep Q-Networks память уже использовалась для дополнительного обучения Q-сети [8]; Neural Episodic Control сочетал обучаемое представление состояния с таблицей оценок ценности [9]. Обе работы цитируются в MacroHFT. Исследовательский вопрос: какую проблему торгового гиперагента решает конкретная организация памяти и чем объясняется её преимущество?

2.6.2. Устройство и работа памяти

Обозначим Q-функцию гиперагента через $Q_\theta(s, a)$, где θ — параметры обучаемой верхней сети. Как и прежде, верхняя сеть формирует её, взвешивая оценки низкоуровневых экспертов. Память хранится отдельно от параметров сети:

$$\mathcal{M}_{\text{mem}} = (K, E, V). \tag{24}$$

В K находятся ключи для поиска, в E — состояния и действия, в V — сохранённые оценки ценности действий.

Добавление перехода. Для наблюдённого перехода (s_j, a_j, r_j, s'_j) кодировщик вычисляет ключ

$$h_j = \psi_{\theta_j}(s_j), \tag{25}$$

где ψ_{θ_j} — кодировщик состояния гиперагента на момент создания записи. Для нетерминального перехода сохраняемое значение имеет вид

$$v_j = r_j + \gamma \max_{a'} Q_{\theta_j}(s'_j, a'). \tag{26}$$

В терминальном состоянии код приравнивает будущую награду к нулю.

v_j — не наблюдённая прибыль всей последующей сделки или завершённого эпизода. Непосредственно наблюдается только r_j; остальную часть сеть вычисляет по собственной Q-оценке. Это одношаговый bootstrap-ориентир, сохранённый для последующего обучения.

Таблица имеет ограниченную ёмкость. При переполнении код удаляет самую старую запись по правилу FIFO. Так память сохраняет недавние переходы, а не постоянный архив наиболее опасных рыночных событий.

Поиск похожих состояний. Для состояния s код вычисляет текущий ключ $h = \psi_\theta(s)$, а затем выбирает k ближайших сохранённых ключей по евклидову расстоянию. Обозначим множество их индексов через $\mathcal{N}_k(h)$.

Опубликованная реализация использует ядро

$$\kappa(h, h_j) = \frac{1}{\|h - h_j\|_2^2 + \varepsilon}, \qquad \varepsilon > 0. \tag{27}$$

Ближайшие записи получают больший вес. Затем реализация учитывает среди найденных соседей только записи с тем же действием:

$$\omega_j(s, a) = \frac{\kappa(h, h_j)\mathbf{1}\{a_j = a\}}{\sum_{\ell \in \mathcal{N}_k(h)} \kappa(h, h_\ell)\mathbf{1}\{a_\ell = a\}}. \tag{28}$$

Дополнительная оценка равна

$$Q_M(s, a) = \sum_{j \in \mathcal{N}_k(h)} \omega_j(s, a) v_j. \tag{29}$$

Формулы (27)–(28) соответствуют опубликованному коду [12]. В формуле (7) статьи вместо обратного расстояния записано само расстояние. Это различие меняет относительный вклад соседей; далее мы используем реализованный вариант. До заполнения таблицы или при отсутствии подходящего действия среди найденных соседей код подставляет обычную одношаговую оценку вместо Q_M.

Обучение и исполнение политики. Функция потерь включает полученный ориентир:

$$\begin{aligned} \mathcal{L}(\theta) &= \mathcal{L}_0(\theta) + \lambda_M \mathcal{L}_M(\theta), \\ \mathcal{L}_M(\theta) &= \mathbb{E}\left[\left(Q_\theta(s, a) - Q_M(s, a)\right)^2\right]. \end{aligned} \tag{30}$$

Здесь $\lambda_M \geq 0$ задаёт силу дополнительного обучения, а $\mathcal{L}_0$ объединяет остальные слагаемые исходного метода, включая

TD-ошибку и вспомогательный DP-ориентир. Реализация рассчитывает Q_M при сборе перехода и сохраняет вместе с ним в обычном replay-buffer.

Во время тестовой торговли модель не выполняет поиск в памяти: обученные сети экспертов и гиперагента определяют действие [12]. Память влияет на результат через полученные при обучении параметры. Поэтому объяснение «агент увидел обвал, вспомнил похожий случай и вышел из позиции» не описывает исполнение этой модели.

Улучшение реакции возможно, если дополнительное обучение сформировало более подходящую политику. Но скорость усвоения переходов при обучении и скорость выхода из позиции на новом рынке — разные свойства. Первое не устанавливает второе.

2.6.3. Опубликованные результаты

Авторы проверяют пользу памяти абляцией *w/o-MEM*. Для памяти они приводят прямое сравнение полной системы с вариантом без этого компонента; для волатильностной специализации и мягкой агрегации такого сравнения нет.

Таблица 3 воспроизводит результаты Table 3 из [1]. В рамках нашего разбора мы не переобучали гиперагента; таблица содержит опубликованные результаты, а не наш эксперимент с памятью.

С памятью доходность и Sharpe растут, а MDD снижается на всех четырёх активах. На ETH разница доходностей составляет 26.55 процентного пункта, на LTC — 26.82. Общее возражение против сложности архитектуры не отменяет этот положительный результат.

Однако таблица не разделяет возможные причины улучшения. В ней нет распределения эффекта по повторным обучениям, а тестовые периоды четырёх активов существенно пересекаются по календарю. Поэтому таблица не даёт оценки устойчивости прибавки к случайной инициализации или смене рыночного периода.

Обсуждая Figure 6, авторы связывают разницу между моделями с несвоевременной реакцией варианта без памяти на падение ETH. Это иллюстрация поведения двух обученных политик. Она не заменяет сравнения на систематически выделенной выборке стрессовых событий и не показывает, какую часть общей прибавки создаёт именно защита от редких потерь.

2.6.4. Какую проблему решает этот механизм

Мотивация памяти объединяет несколько возможных недостатков. Редкие события могут получать мало обучающих обновлений, а Q-оценки могут быть нестабильны. Кроме того, сама цель максимизации средней награды может допускать неприемлемый риск. Эти проблемы требуют разных решений. Абляция «с памятью / без памяти» не позволяет выбрать между соответствующими объяснениями её эффекта.

Недостаточное использование редких переходов. Если проблема — распределение обучающих примеров, менять нужно частоту их использования. Prioritized Experience Replay повышает вероятность повторной выборки переходов с большой TD-ошибкой и использует importance-sampling-поправки для контроля смещения [10]. В финансовой задаче можно проверить и событийную выборку: заранее выделять на train резкие движения или крупные потери и обеспечивать их представительство в обучающих пакетах.

PER не гарантирует особого внимания именно к финансовым катастрофам: большая TD-ошибка может возникать и у обычного шумного перехода. Но этот подход непосредственно проверяет гипотезу о том, что стандартное обучение неудачно распределяет обновления между наблюдениями.

В MacroHFT ни сохранение записи, ни её последующее извлечение не зависят непосредственно от размера потери или редкости рыночного события. FIFO отдаёт предпочтение недавним наблюдениям, а поиск — близким состояниям. Редкий эпизод может быть вытеснен из таблицы; пока он хранится, его вклад зависит от того, насколько часто он оказывается подходящим соседом.

Это не означает, что сведения об эпизоде полностью исчезают: они могут сохраняться в параметрах сети и обычном replay-buffer. Но специальный механизм долговременного сохранения или приоритизации кризисов из такого устройства памяти не следует.

Кроме того, увеличение доли кризисных примеров без коррекции их весов изменяет распределение, относительно которого обучается модель. Если это делается ради более осторожного поведения, такое изменение следует формулировать как часть критерия обучения, а не только как устранение нехватки наблюдений.

Стабилизация и сглаживание Q-оценок. Фактическая функция потерь (30) допускает другое объяснение: память создаёт второй ориентир, полученный усреднением сохранённых оценок. Это может уменьшать разброс обучающих целей и изменять ход оптимизации даже на обычных, неэкстремальных состояниях.

Алгебраически это видно, если для одного обновления зафиксировать TD-ориентир y и значение памяти $z = Q_M(s, a)$. Изолированная квадратичная часть ошибки имеет вид

$$\ell(q) = (q - y)^2 + \lambda_M (q - z)^2, \qquad q = Q_\theta(s, a). \tag{31}$$

Её минимум достигается при

$$q_{\text{eff}} = \frac{y + \lambda_M z}{1 + \lambda_M}. \tag{32}$$

То есть дополнительный член сдвигает целевую оценку к сохранённым значениям. Это свойство двух квадратичных слагаемых, а не решение всей задачи обучения сети.

Такое сглаживание полезно, если соседние записи дают дополнительную информацию об одной условной ценности. Если же близкие эмбеддинги объединяют состояния с разными последствиями действий, усреднение может скрыть нужное различие. Особенно чувствителен этот вопрос на границе рыночных режимов, где от политики как раз требуется быстро изменить решение.

Исходное наблюдение «у похожих состояний сильно различаются награды» не определяет причину разброса. Это может быть случайный шум, ошибка оценки или недостаток информации во входном состоянии. В первом случае усреднение может помочь; в последнем оно не восстанавливает отсутствующий признак, по которому следовало бы различать ситуации.

Сохранённые значения v_j также содержат прогнозы самого гиперагента. Память не предоставляет независимой проверки его долгосрочных оценок. Она может сглаживать ошибки, но может и переносить систематическую ошибку между похожими состояниями. Сами bootstrap-оценки — не недостаток: они используются и в обычном Q-learning. Однако их наличие ограничивает интерпретацию таблицы как хранилища проверенных правильных решений.

Для этой гипотезы естественные конкуренты — другие дополнительные Q-ориентиры, усреднение оценок и регуляризация. Сравнение с ними показало бы, нужны ли именно поиск

Таблица 3: Абляция памяти в MacroHFT по Table 3 [1]. TR — доходность, ASR — годовой коэффициент Шарпа, MDD — максимальная просадка. Все значения взяты из статьи.

	Без памяти			Полная MacroHFT		
Рынок	TR, %	ASR	MDD, %	TR, %	ASR	MDD, %
BTC	2.03	0.49	6.49	3.03	0.61	5.41
ETH	12.73	1.26	20.87	39.28	3.89	9.67
DOT	2.41	0.34	27.23	13.79	0.97	15.89
LTC	-8.66	-0.58	22.03	18.16	1.50	14.24

соседей и отдельная таблица, или основной эффект даёт дополнительное стабилизирующее слагаемое.

Отношение политики к риску. Даже точное запоминание всех кризисных переходов не обязывает агента избегать рискованной стратегии. Если она имеет положительную ожидаемую награду, максимизация математического ожидания может предпочесть её более безопасной альтернативе.

Функция потерь MacroHFT включает ошибку согласования Q-оценок, но не явное ограничение на хвостовые потери. Поэтому снижение просадки в таблице — эмпирический результат обучения, а не прямое следствие оптимизируемого риск-критерия.

Если цель — ограничить редкие крупные убытки, задачу можно поставить явно. Например, для совокупной награды G_π рассмотреть

$$J_{\text{risk}}(\pi) = \mathbb{E}[G_\pi] - \lambda_R \, \text{CVaR}_\alpha(-G_\pi), \tag{33}$$

где $\lambda_R \geq 0$, а CVaR_α характеризует среднюю потерю в верхнем хвосте распределения убытков; при $\alpha = 0.95$ это худшие 5% исходов. Risk-sensitive постановки с CVaR исследовались до MacroHFT [11].

Формула (33) задаёт альтернативную задачу, а не описывает скрытое действие памяти. Она требует сравнения по компромиссу между доходностью и риском. Память может улучшать оценивание наград, а риск-чувствительная цель — менять предпочтения политики; эти механизмы способны дополнять друг друга.

2.6.5. Что следует проверить дальше

Сначала нужно решить, какое объяснение проверять. Изменение выборки, сглаживание Q-targets и изменение отношения к риску — не взаимозаменяемые абляции.

Чтобы проверить работу с редкими событиями, нужно сравнить обычный replay, PER, событийную выборку и память MacroHFT при одних и тех же низкоуровневых экспертах и гиперагенте. Кроме итогового PnL, нужно измерить, как часто кризисные переходы участвуют в обучении, какие записи извлекает память и уменьшаются ли потери на новых стрессовых эпизодах. Без этого объяснение через редкие события остаётся предположением.

Чтобы проверить роль поиска, нужен контроль с тем же дополнительным слагаемым ошибки, но другим способом построения Q_M. Можно сравнить ближайшие состояния со случайными записями того же действия и сопоставимого возраста, а также с равномерным усреднением найденных соседей. Это отделит выбор релевантных переходов от самого наличия второго обучающего ориентира. Дополнительно следует оценивать качество Q-оценок на отложенных данных: снижение ошибки на собственном bootstrap-target ещё не устанавливает улучшения прогноза награды.

Если объяснять эффект управлением риском, нужны сравнения с явно заданным риск-критерием. Оценивать следует хвостовые потери, просадки и пропущенную доходность, а не выбирать победителя только по общей прибыли. Также нужно показать различия рыночной экспозиции: снижение убытков за счёт более редкого нахождения в позиции не равнозначно улучшению распознавания опасных состояний.

Во всех сравнениях необходимо сохранять одинаковые обучающие данные, архитектуру гиперагента и сопоставимые бюджеты настройки и обновлений. Время обучения и стоимость поиска в памяти следует учитывать отдельно. Повторные инициализации и непересекающиеся тестовые периоды позволят отличить устойчивый эффект от особенностей выбранных обученных политик.

Опубликованная абляция поддерживает полезность добавленного модуля в конкретной системе, но не устанавливает, какой недостаток обучения он устраняет. Обоснование памяти как средства работы с экстремальными событиями требует именно такого разделения механизмов. Иначе высокая доходность подтверждает удачное сочетание компонентов. Но она не объясняет, почему для той же задачи следует предпочесть эпизодическую память уже существующим способам использования опыта и управления риском.

Список литературы

[1] C. Zong, C. Wang, M. Qin, L. Feng, X. Wang, and B. An. MacroHFT: Memory Augmented Context-aware Reinforcement Learning On High Frequency Trading. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 4712–4721. DOI: 10.1145/3637528.3672064.

[2] M. Qin, S. Sun, W. Zhang, H. Xia, X. Wang, and B. An. EarnHFT: Efficient Hierarchical Reinforcement Learning for High Frequency Trading. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(13), 2024, pp. 14669–14676. DOI: 10.1609/aaai.v38i13.29384.

[3] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai. Deep Direct Reinforcement Learning for Financial Signal Representation and Trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28(3), 2017, pp. 653–664 (published online in 2016). DOI: 10.1109/TNNLS.2016.2522401.

[4] T. Théate and D. Ernst. An Application of Deep Reinforcement Learning to Algorithmic Trading. *Expert Systems with Applications*, 173, 2021, article 114632. DOI: 10.1016/j.eswa.2021.114632.

[5] T. Zhu and W. Zhu. Quantitative Trading through Random Perturbation Q-Network with Nonlinear Transaction Costs. *Stats*, 5(2), 2022, pp. 546–560. DOI: 10.3390/stats5020033.

[6] R. Almgren and N. Chriss. Optimal Execution of Portfolio Transactions. *The Journal of Risk*, 3(2), 2001, pp. 5–39. DOI: 10.21314/JOR.2001.041.

[7] D. Byrd, M. Hybinette, and T. H. Balch. ABIDES: Towards High-Fidelity Multi-Agent Market Simulation.

Proceedings of the ACM SIGSIM Conference on Principles of Advanced Discrete Simulation, 2020, pp. 11–22. DOI: 10.1145/3384441.3395986.

[8] Z. Lin, T. Zhao, G. Yang, and L. Zhang. Episodic Memory Deep Q-Networks. arXiv:1805.07603, 2018. https://arxiv.org/abs/1805.07603.

[9] A. Pritzel, B. Uria, S. Srinivasan, A. Puigdomènech Badia, O. Vinyals, D. Hassabis, D. Wierstra, and C. Blundell. Neural Episodic Control. *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70, 2017, pp. 2827–2836. https://proceedings.mlr.press/v70/pritzel17a.html.

[10] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized Experience Replay. *International Conference on Learning Representations*, 2016. https://arxiv.org/abs/1511.05952.

[11] Y. Chow, A. Tamar, S. Mannor, and M. Pavone. Risk-Sensitive and Robust Decision-Making: a CVaR Optimization Approach. *Advances in Neural Information Processing Systems*, 28, 2015, pp. 1522–1530. https://arxiv.org/abs/1506.02188.

[12] ZONG0004. MacroHFT: official implementation. Рассмотренная версия: commit `31e5ef41f93b2aea6e63e6ae2267675c997a8e54`. Файлы `RL/util/memory.py` и `RL/agent/high_level.py`. https://github.com/ZONG0004/MacroHFT/tree/31e5ef41f93b2aea6e63e6ae2267675c997a8e54.